

Graph Problem Bank

Legend

★ = Core Placement Problem (Mandatory)

Language-Agnostic DSA Problem Bank | Build graph representations from scratch. Cover directed and undirected graphs separately.

Section 1: Graph Representation — Internals

- ★ Represent a Graph as an Adjacency Matrix
- ★ Represent a Graph as an Adjacency List
- ★ Add an Edge (Directed and Undirected)
- ★ Remove an Edge from a Graph

Add and Remove a Vertex

- ★ Display All Edges of a Graph

Convert Adjacency Matrix to Adjacency List

Convert Adjacency List to Adjacency Matrix

Section 2: Graph Traversal — BFS and DFS

- ★ Breadth First Search (BFS) — Iterative using Queue
- ★ Depth First Search (DFS) — Recursive
- ★ Depth First Search (DFS) — Iterative using Stack
- ★ BFS on Disconnected Graph
- ★ DFS on Disconnected Graph

Print All Nodes at Distance K from Source (BFS)

Section 3: Cycle Detection

- ★ Detect Cycle in Undirected Graph using BFS
- ★ Detect Cycle in Undirected Graph using DFS
- ★ Detect Cycle in Directed Graph using DFS

Detect Cycle in Directed Graph using Kahn's Algorithm (BFS)

Find the Cycle and Print its Nodes

Section 4: Shortest Path Algorithms

- ★ Shortest Path in Unweighted Graph using BFS
- ★ Dijkstra's Algorithm (Single Source Shortest Path)
- Bellman-Ford Algorithm (Handles Negative Weights)
- ★ Floyd-Warshall Algorithm (All Pairs Shortest Path)

Shortest Path in a DAG using Topological Sort

Find the Number of Paths between Two Nodes

Section 5: Topological Sort

- ★ **Topological Sort using DFS**
- ★ **Topological Sort using Kahn's Algorithm (BFS / Indegree)**
- ★ **Course Schedule Problem (Detect Cycle in Prerequisites)**

Course Schedule II (Return a Valid Order)

Alien Dictionary (Determine Character Order)

Section 6: Connected Components

- ★ **Count Connected Components in an Undirected Graph**
- ★ **Check if a Graph is Connected**
- Find All Connected Components (List Each Component)
- ★ **Strongly Connected Components using Kosaraju's Algorithm**

Bridges in a Graph (Critical Connections)

Articulation Points in a Graph

Section 7: Minimum Spanning Tree

- ★ **Prim's Algorithm for Minimum Spanning Tree**
- ★ **Kruskal's Algorithm using Union-Find**
- ★ **Implement Union-Find (Disjoint Set Union) with Path Compression**

Minimum Cost to Connect All Points

Check if Two Nodes are in the Same Component (Union-Find)

Section 8: Advanced Graph Patterns

- ★ **Number of Islands (Grid BFS/DFS)**
- ★ **Clone a Graph**
- ★ **Bipartite Graph Check (2-Colorable using BFS)**
- ★ **Find if Path Exists between Source and Destination**

Word Ladder (BFS on Implicit Graph)

Network Delay Time (Dijkstra)

Cheapest Flight within K Stops (Modified BFS/Bellman-Ford)

Section 9: Real-World Applications of Graphs

- ★ **GPS Navigation / Road Map — Find Shortest Route using Dijkstra**
- ★ **Social Network Friend Suggestion — Find Friends of Friends using BFS**
- ★ **Task Dependency Scheduler — Order Tasks using Topological Sort**
- Web Crawler — Crawl and Index Pages using BFS on Hyperlink Graph
- ★ **Network Topology Analyser — Find Critical Connections (Bridges)**
- Recommendation System — Find Similar Users using Graph Similarity
- ★ **Deadlock Detection in OS — Detect Cycle in Resource Allocation Graph**
- Flight Route Optimizer — Find Cheapest Path with Layover Constraints